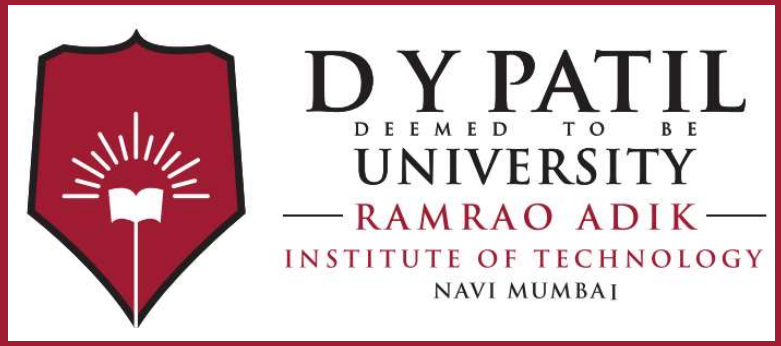


Subject Name: OOP

Unit No:1

Unit Name: Introduction to OOP

Index

Lecture 4 – Features of Java Language, JDK environment	**3**
Lecture 5 – Basic Constructs of Java Programming	13
Lecture 6 – Input and output functions in Java : Buffered reader class, Scanner class	39



D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 4
Features of Java Language, JDK environment



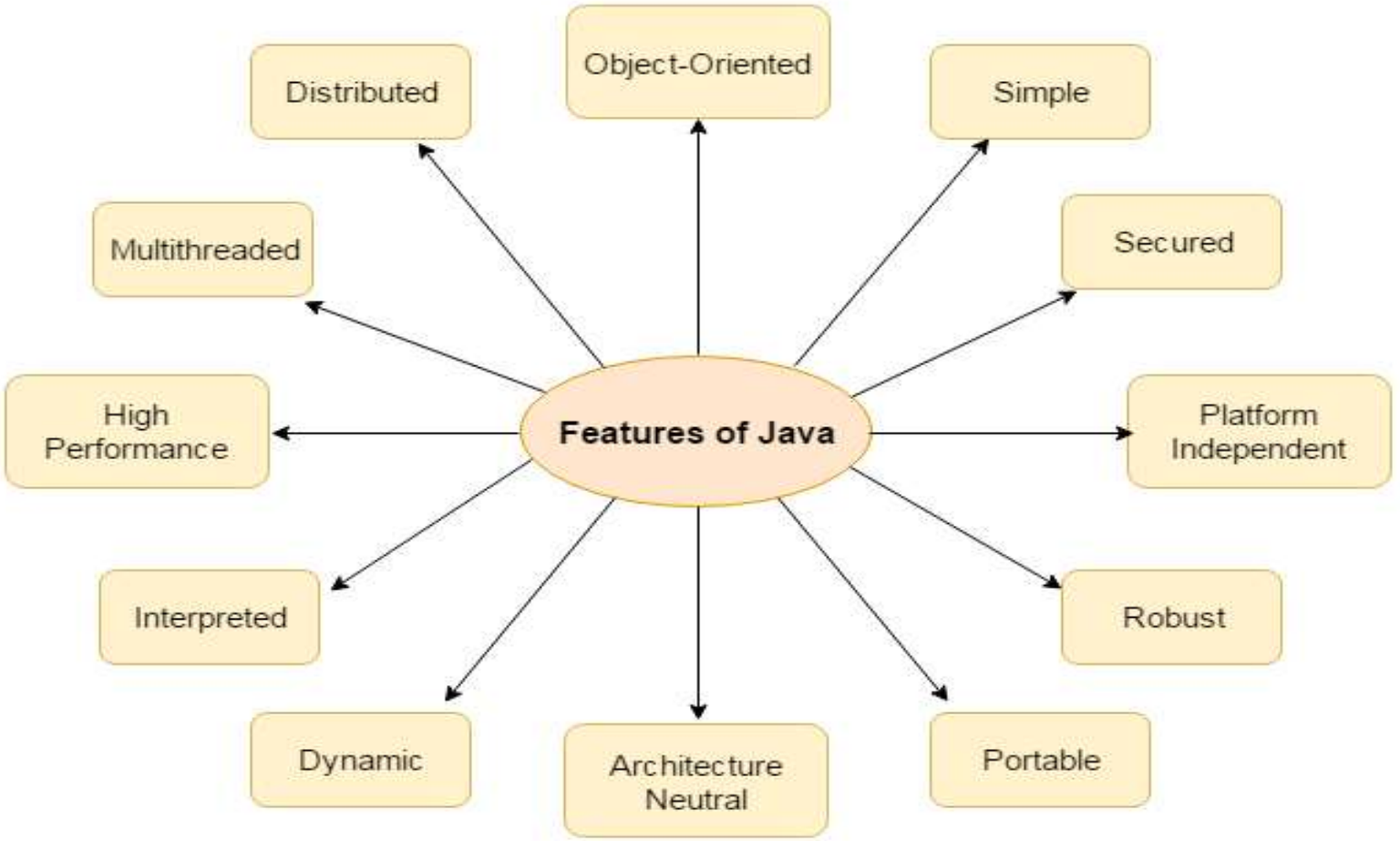
Introduction to JAVA

- A general purpose object oriented programming language developed by Sun Microsystems in 1991.
- Originally called Oak by **James Gosling**, the first designer of Java and implemented its original compiler and virtual machine.
- Renamed as **JAVA** in 1995
- Versions
  - Java 2 Micro Edition (J2ME)
  - Java 2 Standard Edition (J2SE)
  - Java 2 Enterprise Edition (J2EE)



D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features of Java



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Features of Java

- **Simple:**
  - Easy to learn.
  - removed many confusing and rarely-used features

    e.g., explicit pointers, operator overloading etc.

- **Object-oriented:**
  - Organize our software as a combination of different types of objects that incorporates both data and behaviour.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Features of Java

- **Platform Independent:**
  - A platform is the hardware or software environment in which a program runs.
  - It has two components:
    - ✓ Runtime Environment
    - ✓ API(Application Programming Interface)
  - "Write-Once Run-Anywhere"
    - ✓ Java code can be run on multiple platforms e.g. Windows, Linux, Sun Solaris, Mac/OS etc.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Features of Java

- **Robust:**
  - Strong memory management.
  - No usage of pointers.
  - Exception handling and type checking mechanisms.

"An Exception is the condition that is caused by run time error in the program. If it occurred java interpreter creates an exception object and throws it."

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Features of Java

- **Multi-threaded**
  - A thread is like a separate program, executing concurrently.
  - Java programs deal with many tasks at once by defining multiple threads.
  - Doesn't occupy memory for each thread.
  - Shares a common memory area.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Features of Java

- **Distributed and Network Oriented**

  Java grew up in the days of the Internet

  - Inherently network friendly
  - Original release of Java came with Networking libraries
  - Newer releases contain even more for handling distributed applications

    - RMI, Transactions

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

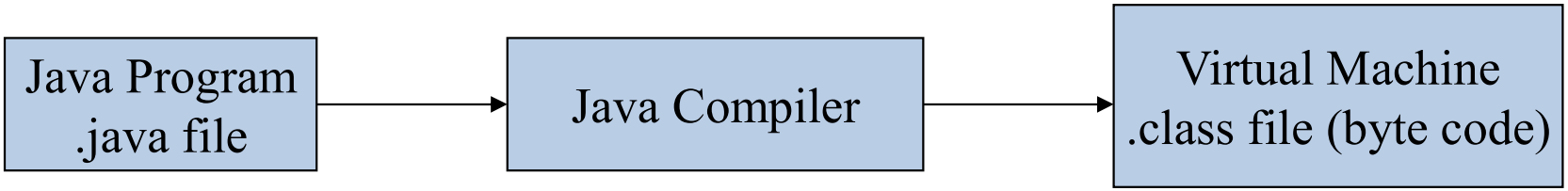
Features of Java

- **Compiled and Interpreted**
  - Java works in two stage
    - **Java compiler** translate the source code into byte code.
    - **Java interpreter** converts the byte code into machine level representation.
  - **Byte Code:**
    - A highly optimized set of instructions to be executed by the java runtime system, known as java virtual machine (JVM).
    - Not executable code.
  - **Java Virtual Machine (JVM):**
    - Need to be implemented for each platform.
    - Although the details vary from machine to machine, all JVM understand the same byte code.

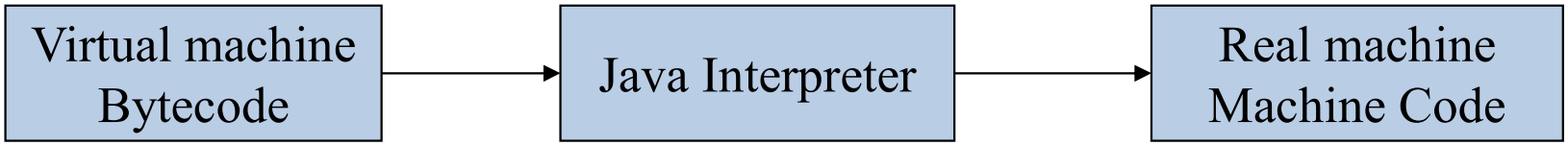
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Virtual Machine

- ✓ Java compiler produces an intermediate code known as byte code for a machine, known as JVM.
- ✓ It exists only inside the computer memory.



- ✓ Machine code is generated by the java interpreter by acting as an intermediary between the virtual machine and real machine.



D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Module No 1: Introduction to OOP

Lecture No: 5
Basic Constructs of Java Programming



Simple Java Program

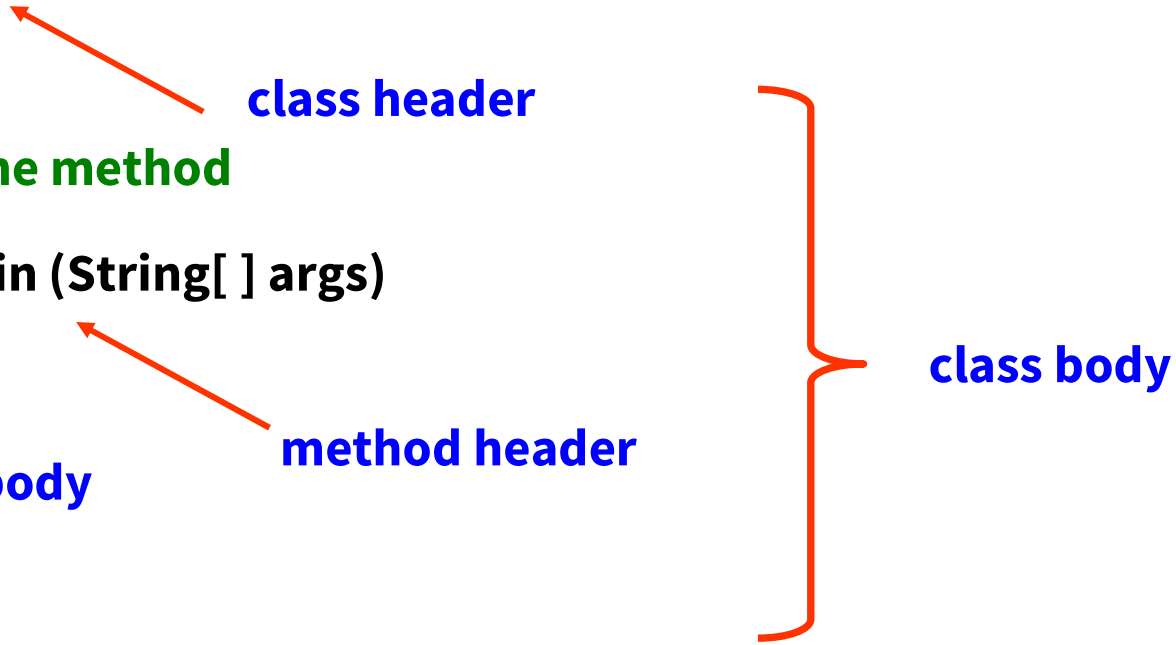
```
// Program to print Hello message
public class Hello
{
    public static void main (String[ ] args)
    {
        System.out.print ("Hello");
    }
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Program Structure

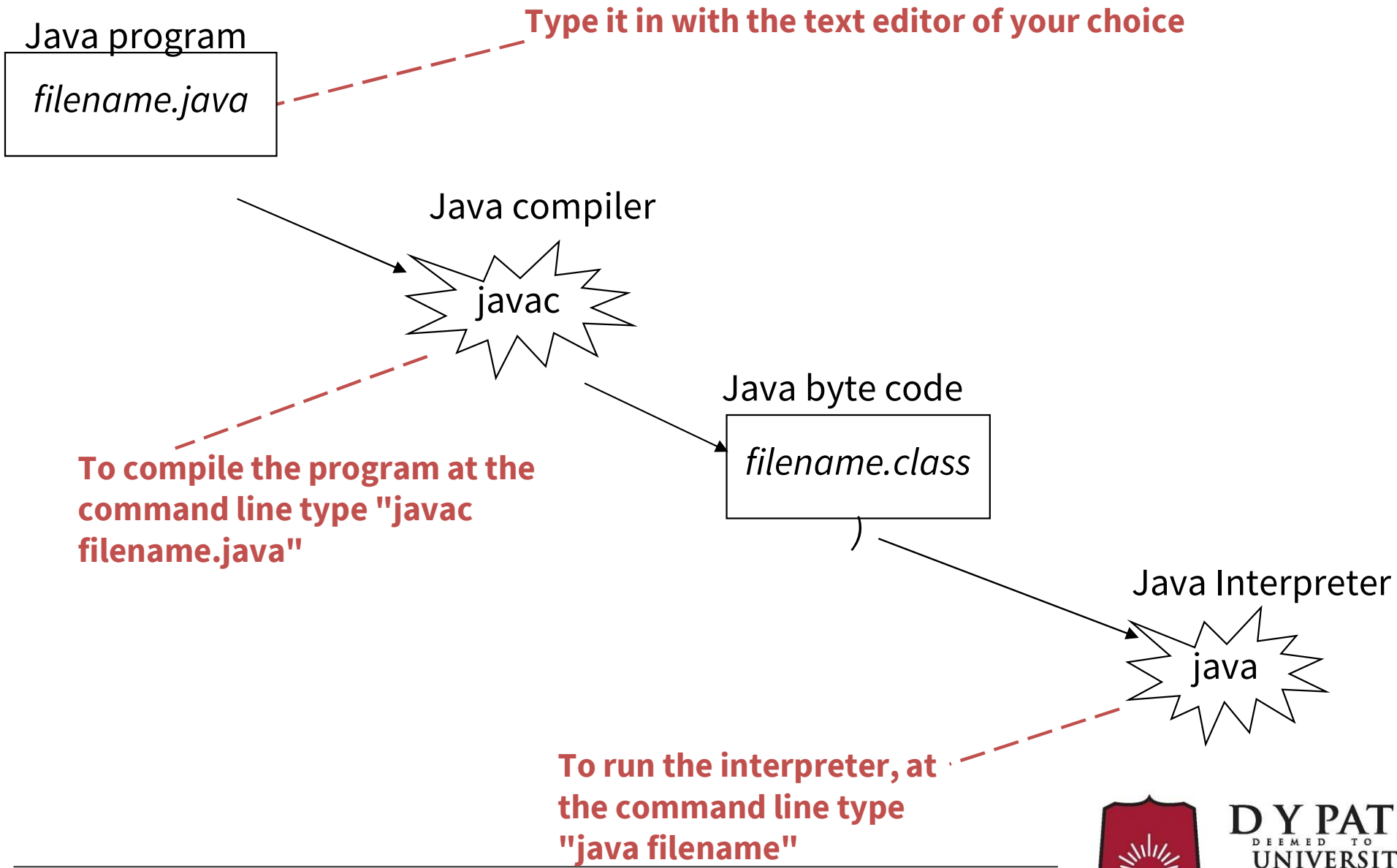
```
// comments about the class
public class MyProgram
{
    // comments about the method
    public static void main (String[ ] args)
    {

    }
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Compile and Execute Java Program



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Basic Constructs of Java

- Constants
- Variables
- Data Types
- Operators and Expression
- Revision of Branching and Looping

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Constants

- The variable whose value is always constant (does not change).
- The reserved word **final** is used in constant declaration.
- Syntax:

  final data_type constant_name=constant_value;

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Variables

- A variable can be thought of as a container which holds value for you, during the life of a Java program.
- Every variable is assigned a **data type** which designates the type and quantity of value it can hold.
- All variables must be declared before its use.
- Variables can be declared almost anywhere (scope rules apply)
- Variables have default initialization values
  - Integers: 0
  - Reals: 0.0
  - Boolean: False
- Variables can be initialized in the declaration.
- **Syntax**:

  [access_modifier] [**static**] [**final**] type name = [value];

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Data Types

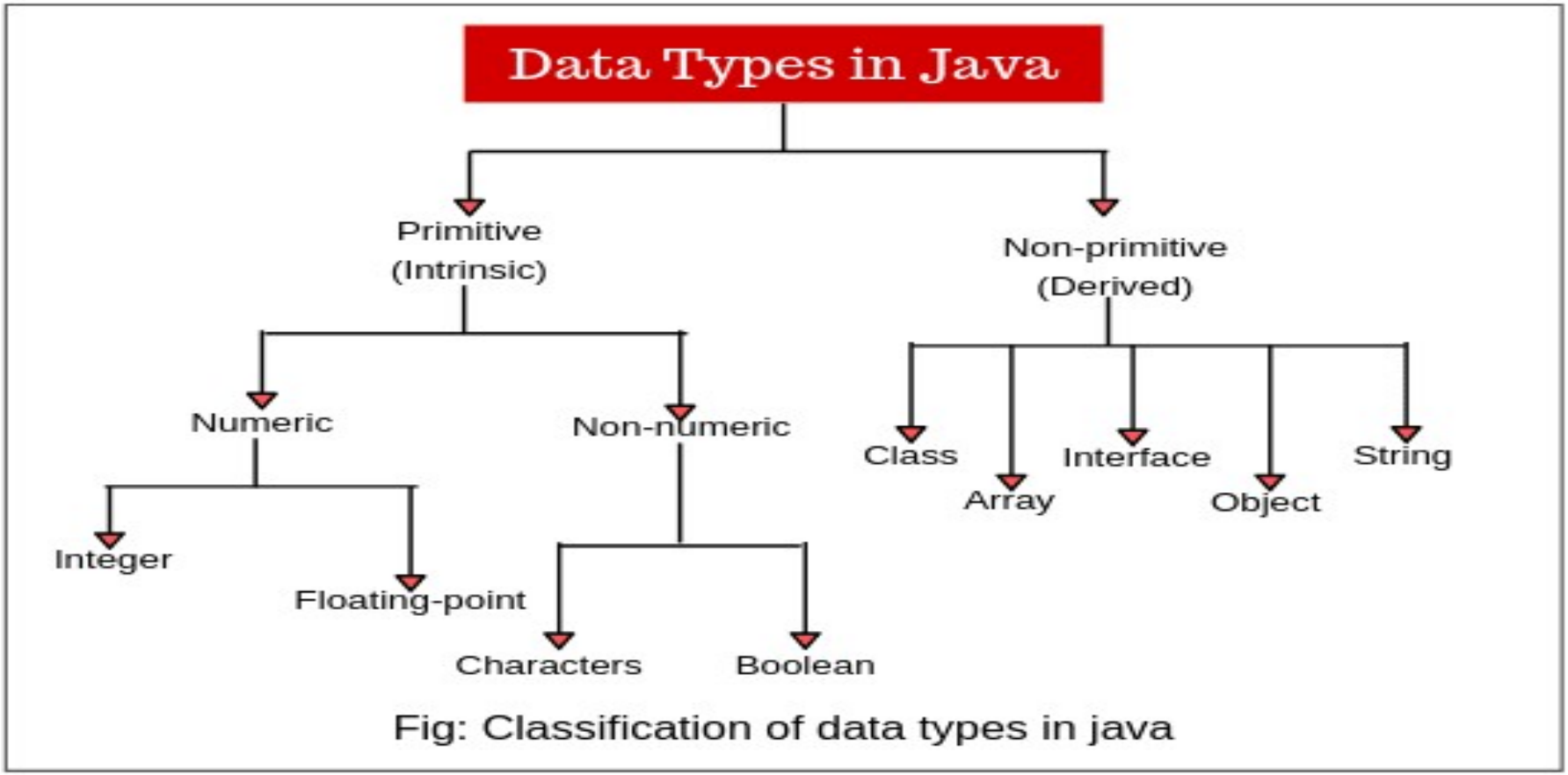


Fig: Classification of data types in java

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Data Types

- **Primitive Data Types**
  - byte (1), -128 to 127
  - short (2), -32768 to 32767
  - int (4), -2147483648 to 2147483647
  - long (8), -9223372036854775808 to 9223372036854775807
  - float (4), -3.4E38 to 3.4E38, 7 digit precision
  - double (8), -1.7E308 to 1.7E308, 17 digits precision
  - char (2), unicode characters
  - boolean (true, false), discrete values

- **Non-primitive/ User-defined Data Types**
  - Array
  - Class
  - interface

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Keywords in Java

- These are the reserved words in JAVA which cannot be used as variable names or identifiers.

abstract	continue	for	new	switch
assert***	default	goto*	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum****	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp**	volatile
const*	float	native	super	while



D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Operators

- Operators/Order of Precedence

Operator	Operation	Precedence
() or [ ] or . or ++ or - -	Parens, array indices, object invocation, increment, decrement	1
+ or – or ~	Unary plus, unary minus, complement	2
* or / or %	Multiplication, division, modulus	3
+ or -	Addition, subtraction	4
<<,>>	Bitwise shifts	5
&	Bitwise AND	6
^	Bitwise XOR	7
|	Bitwise OR	8

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Operators

- Example Usage

```
int anInt1 = 2, anInt2 = 1, anInt3;
double aDbl1;
anInt3 = anInt1 + anInt2 * 4;          // anInt3 gets 6
anInt3 = (anInt1 + anInt2) * 4;        // anInt3 gets 12
anInt3 = ++anInt2 * 2;                 // anInt3 gets 4
anInt2 = 1;
anInt3 = anInt2++ * 2;                 // anInt3 gets 2
anInt2 = 1;
anInt3 = anInt1 & anInt2;              // anInt3 gets 0
anInt3 = anInt1 ^ anInt2;              // anInt3 gets 3
anInt3 = anInt2 << 1;                  // anInt3 gets 2
anInt3 = 1 / 2;                        // anInt3 gets 0
aDbl1 = 1 / 2;                         // aDbl1 gets 0.0
aDbl1 = 1.0 / 2.0;                     // aDbl1 gets 0.5
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Operators.....

- Assignments
  - General Format: variable = expression ;

    Where variable is a previously declared identifier and expression is a valid combo of identifiers, operators, and method (a.k.a. procedure or function) calls
  - Shortcuts:

```
var *= expr ;    // Equivalent to var = var * (expr);
var /= expr ;    // Equivalent to var = var / (expr);
var += expr ;    // Equivalent to var = var + (expr);
var -= expr ;    // Equivalent to var = var - (expr);
var %= expr ;    // Equivalent to var = var % (expr);
var++;           // Equivalent to var = var + 1;
var--;           // Equivalent to var = var - 1;
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Control Statements in Java

1. Conditional Statements
- if-else statements
- switch case statements

2. Iterative Statements
- for loop
- while loop
- do-while loop

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- "if" Statements
  - if with code block

```
if (boolean_expr)
{
      statements
}
```

  - if with single statement

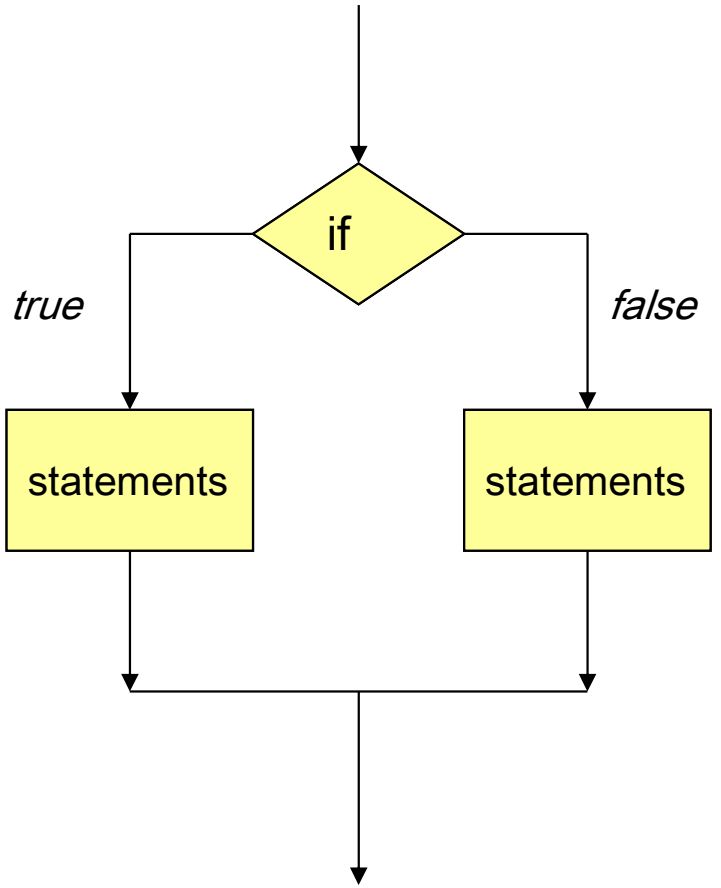
```
if (boolean_expr)
      statement;
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- if" Statements (Continued)
  - if-else

```
if (boolean_expr)
{
    statements for true
}
else
{
    statements for false
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- Boolean Expressions
  - Boolean expressions use conditional operators such that they result in a value of true or false
  - Conditional Operators (Not by order of precedence)

Operators	Operations
== or !=	Equality, not equal
> or <	Greater than, less than
>= or <=	Greater than or equal, less than or equal
!	Unary negation (NOT)
& or &&	Evaluation AND, short circuit AND
| or ||	Evaluation OR, short circuit OR

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Conditional Constructs

- Boolean Expression Examples

```
int i1 = 1, i2 = 2, i3 = 3, i4 = 0;
boolean b1 = true, b2 = false, b3 = true;

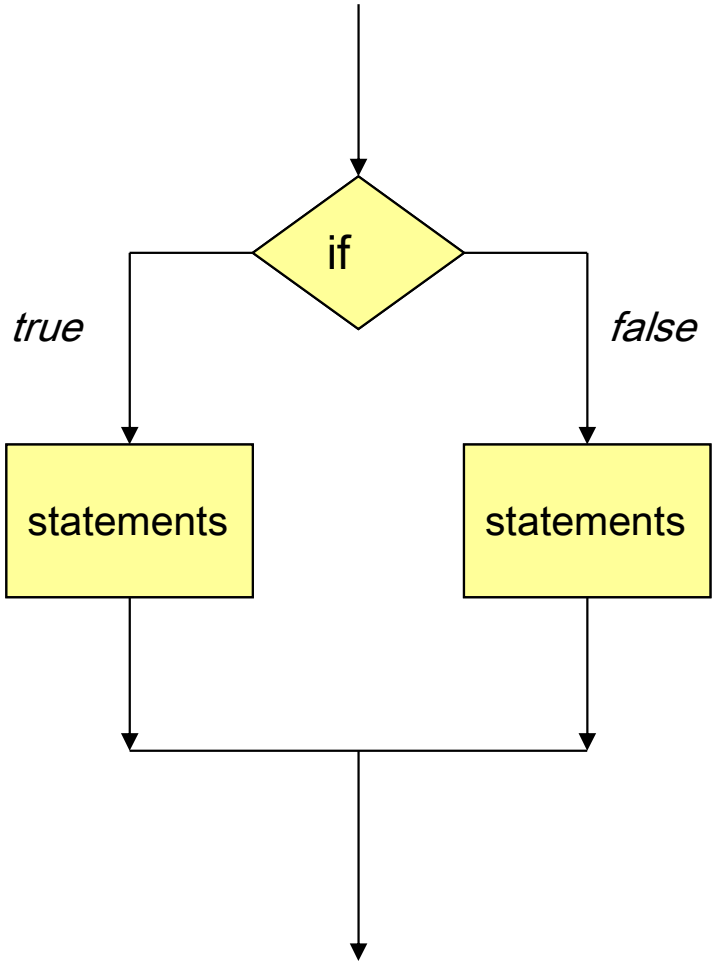
i2 > i1                              // true
(i3 - i2) > i1                       // false
(i3 - i2) >= i1                      // true
(i2 > i1) & (i2 < i3)                // true
(i2 < i1) | (i2 > i1)                // true
i2 != i1                             // true
(i1 < i2) | ((i1/i4) > 1)            // Divide by 0 exception
(i1 < i2) || ((i1/i4) > 1)           // true
(i1 < i2) | (i1++ > 1)               // true, i1 contains 2
(i1 < i2) || (i1++ > 1)              // true, i1 contains 1
b1 && b2 && b3                       // false
(b1 || b2) && b3                     // true
b1 && (i1 == (i3 - i2))              // true
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- if-else" Statement Example

```
class Example
{
    static public void main(String args[])
    {
        // A very contrived example
        int i1 = 1, i2 = 2;
        System.out.print("Result: ");
        if (i1 > i2)
        {
            System.out.println("i1 > i2");
        }
        else
        {
            System.out.println("i2 >= i1");
        }
    }
}
```

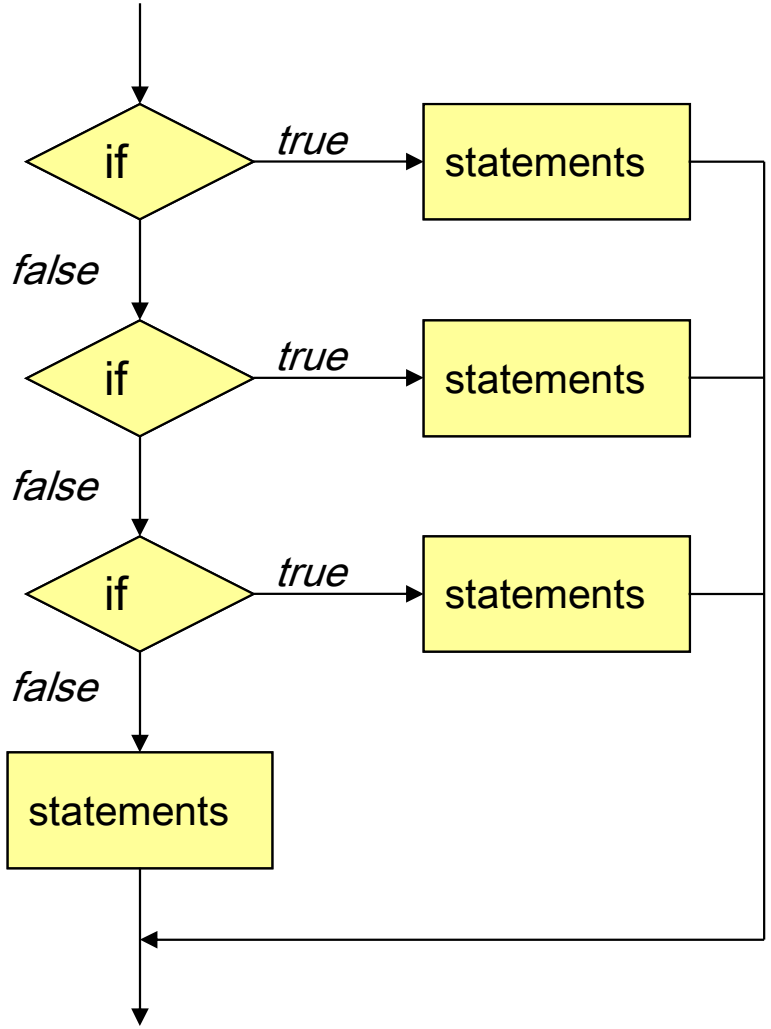


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- The Switch Statement

```
switch (integer_expression)
{
     case int_value_1:
          statements
          break;
     case int_value_2:
          statements
          break;
     …
     case int_value_n:
          statements
          break;
     default:
          statements
}
```

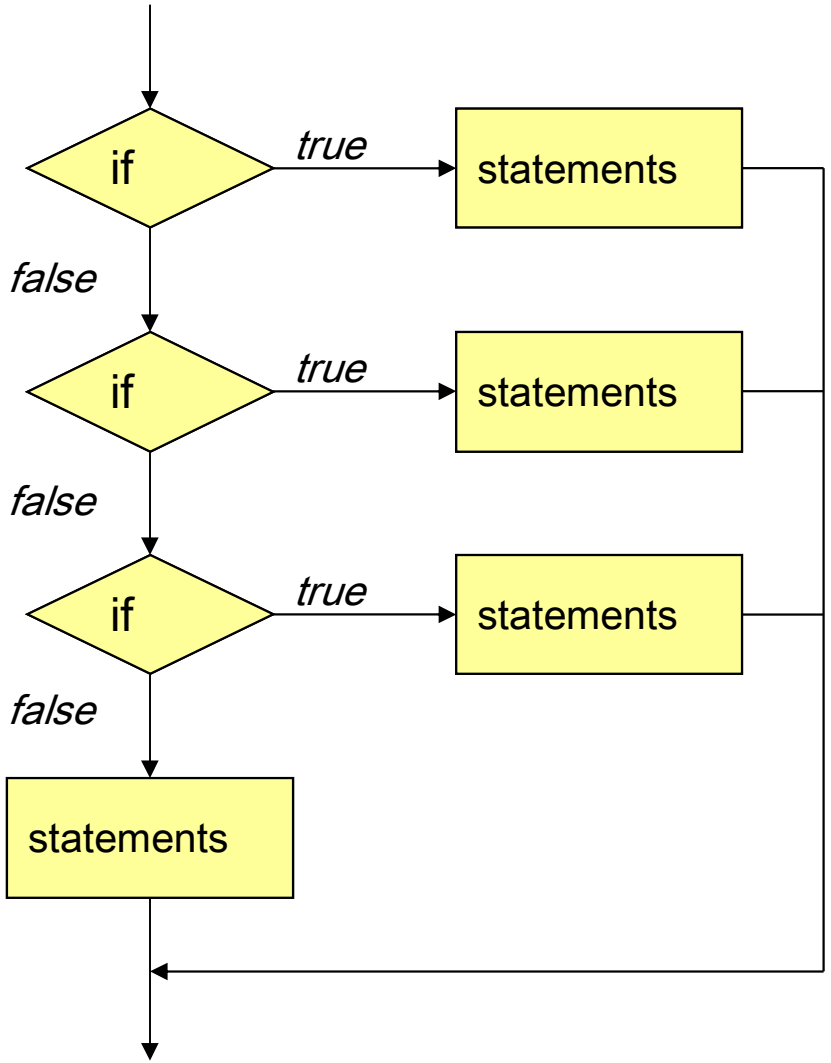


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- Don't forget the "break"

```
switch (integer_expression)
{
     case int_value_1:
          statements
          // No break!
     case int_value_2:
          statements
          break;
     …
     case int_value_n:
          statements
          break;
     default:
          statements
}
```

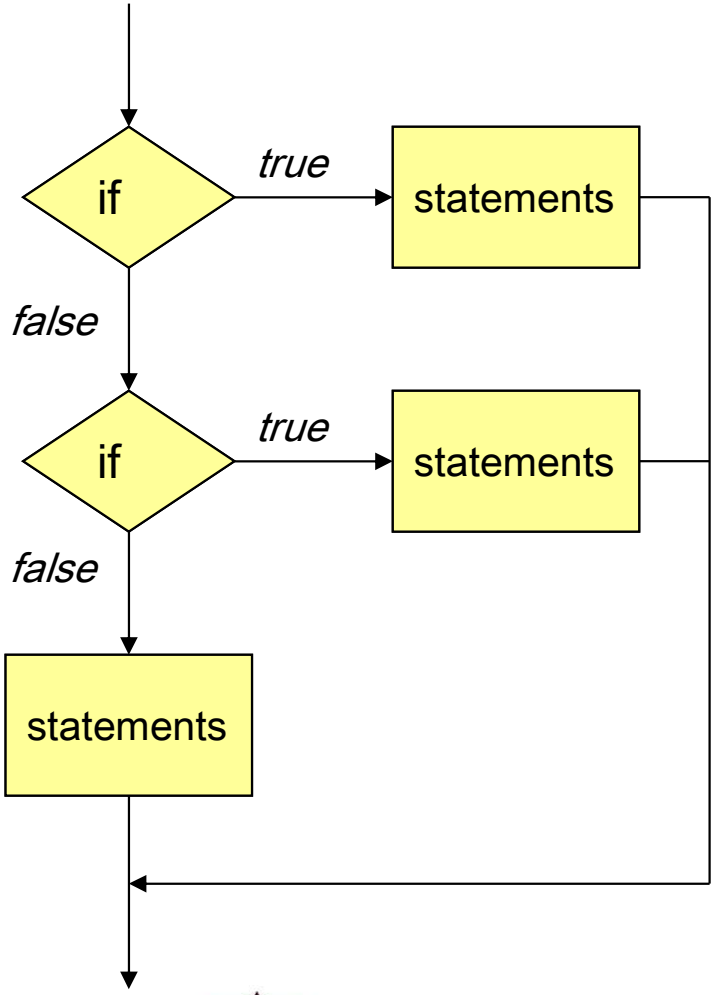


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Conditional Constructs

- Example

```
int n = 5;
switch (n)
{
    case 1:
        n = n + 1;
        break;
    case 5:
        n = n + 2;
        break;
    default:
        n = n - 1;
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Looping Constructs

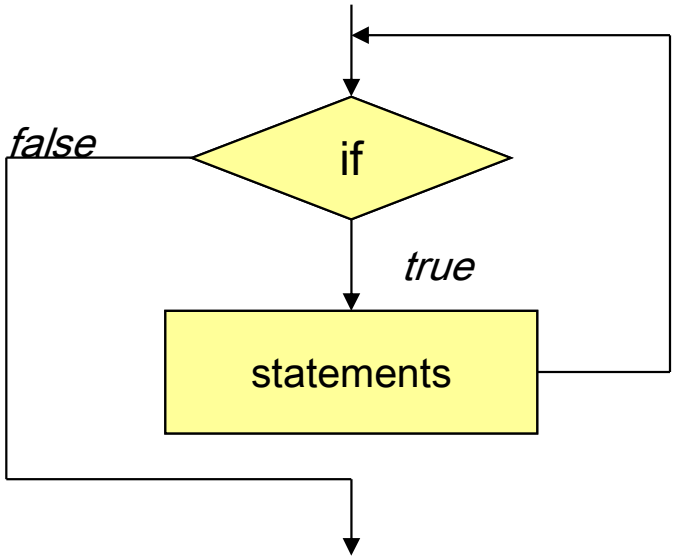
- while loop
  - Exit condition evaluated at top
- do loop
  - Exit condition evaluated at bottom
- for loop
  - Exit condition evaluated at top
  - Includes a initialization statements
  - Includes a update statements for each iteration

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Looping Constructs

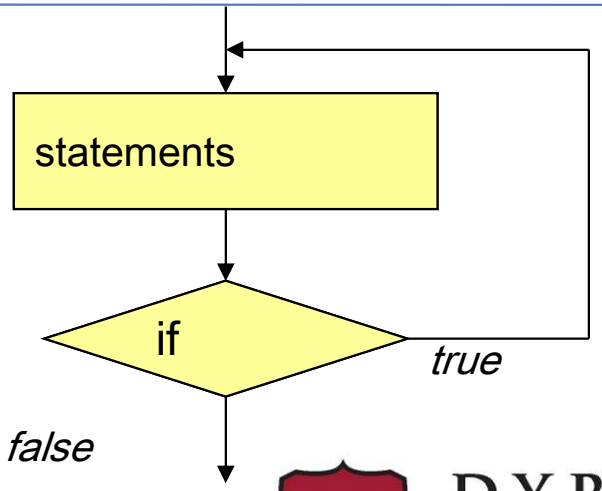
- while loop

```
while (boolean_expr)
{
    statements
}
```



- do loop

```
do
{
    statements
}
while (boolean_expr)
```

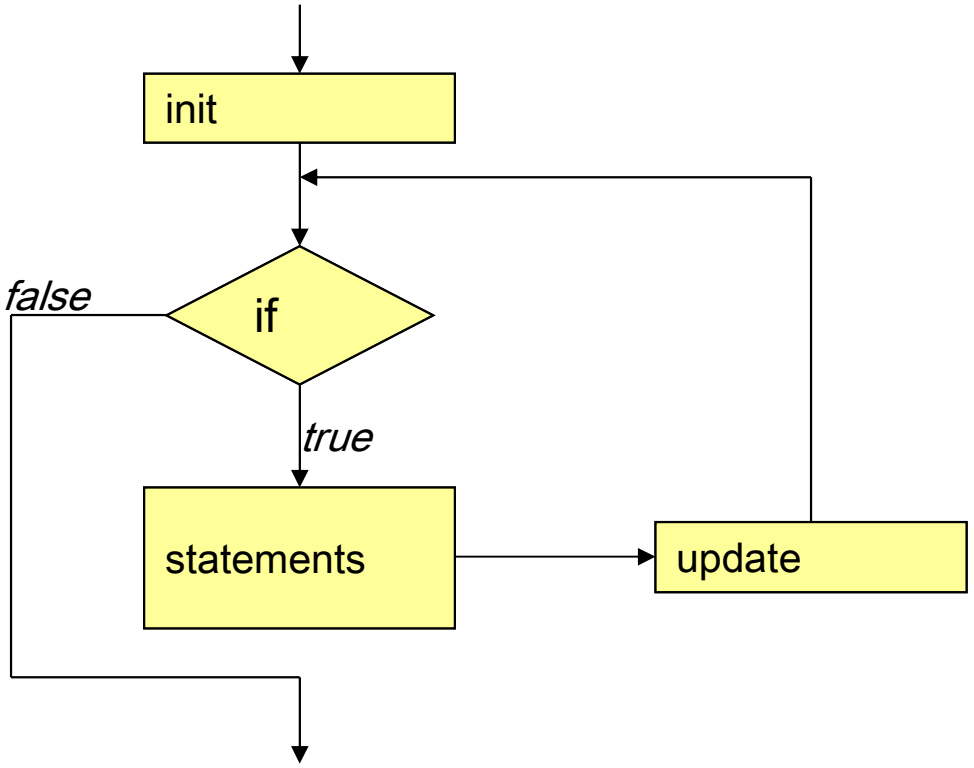


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Looping Constructs

- for loop

```
for (init_stmnt; bool_expr; update_stmnt)
{
    statements
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Looping Constructs

```
class Example
{   static public void main(String args[])
    {
        int i = 0;
        System.out.println("while loop");
        while (i < 10)
        {
            System.out.println(i);
            i++;
        }

        System.out.println("do loop");
        do
        {
            System.out.println(i);
            i--;
        }
        while (i > 0);

       System.out.println("for loop");
        for (i = 0; i < 10; i++)
        {
            System.out.println(i);
        }
    } // End main
} // End Example
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Module No 1: Introduction to OOP

Lecture No: 6
Input and output functions in Java : Buffered reader class, Scanner class



Input/Output

- Java I/O (Input and Output) is used to process the input and produce the output.
- Java uses the concept of a stream to make I/O operation fast.
- The java.io package contains all the classes required for input and output operations.
- We can perform file handling in Java by Java I/O API.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

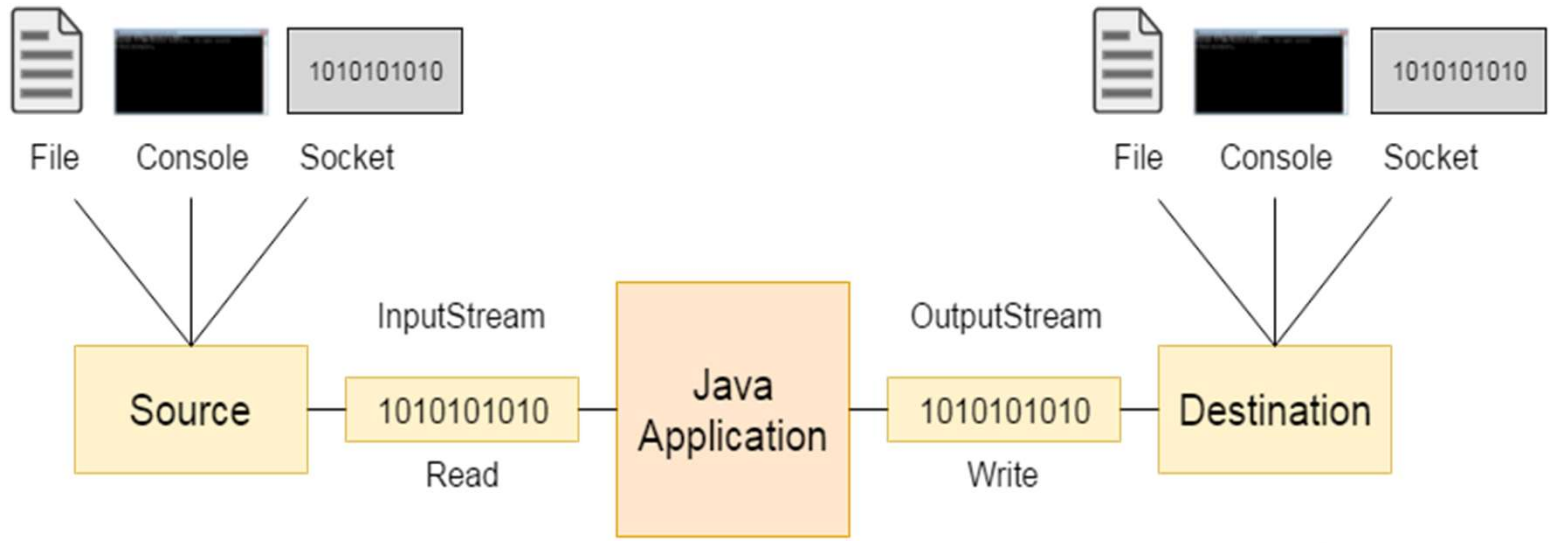
Input/Output

- **Stream -** A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.
- In Java, 3 streams are created for us automatically. All these streams are attached with the console.
  - System.out: standard output stream
  - System.in: standard input stream
  - System.err: standard error stream

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

OutputStream vs InputStream

- **OutputStream -** Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.
- **InputStream -** Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

OutputStream class

- OutputStream class is an abstract class. It is the superclass of all classes representing an output stream of bytes. An output stream accepts output bytes and sends them to some sink.
- **Useful methods of OutputStream**

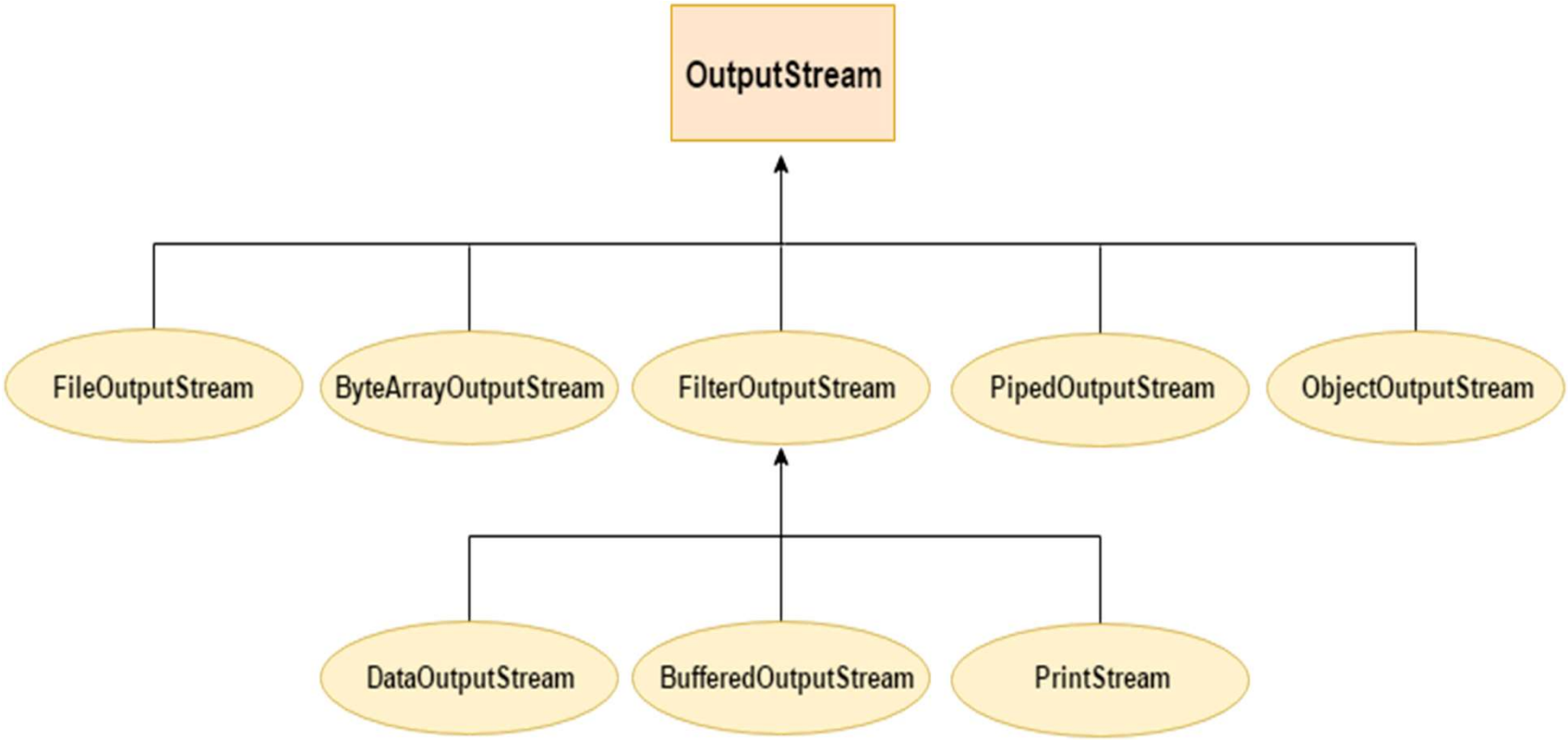
Method	Description
1) public void write(int)throws IOException	is used to write a byte to the current output stream.
2) public void write(byte[])throws IOException	is used to write an array of byte to the current output stream.
3) public void flush()throws IOException	flushes the current output stream.
4) public void close()throws IOException	is used to close the current output stream.



D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

OutputStream class

- **OutputStream Hierarchy**



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

InputStream class

- InputStream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.
- **Useful methods of OutputStream**

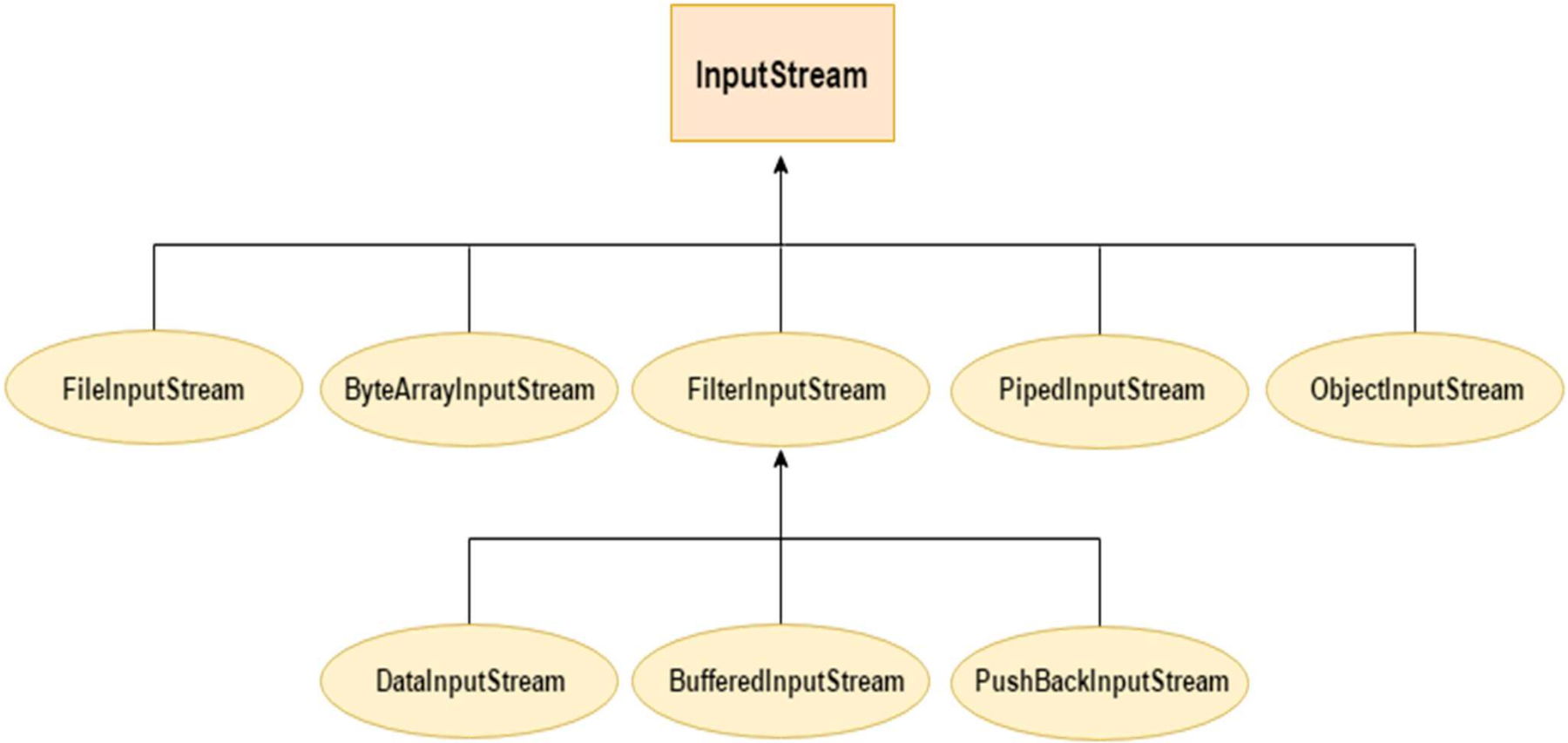
Method	Description
1) public abstract int read()throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
2) public int available()throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
3) public void close()throws IOException	is used to close the current input stream.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

InputStream class

- **InputStream Hierarchy**



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Classes used for Reading Information

- **BufferedReader** (has always been there)
  - **import java.io.*;**
- **Scanner** (specific to JAVA 5 and beyond)
  - **import java.util.*;**



D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

java.io.BufferedReader

- The BufferedReader class can only read Strings
- However we can transform those Strings into numbers of any kind.
- This one is always safe.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Two Objects and one Exception

- In the class:

  public static void main(String args[ ]) **throws IOException**{

  InputStreamReader isr = new InputStreamReader(System.in);

  BufferedReader br = new BufferedReader(isr);

- The method:

  String readLine( )

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Reading a String with BufferedReader

- Reading a String

```
InputStreamReader isr = new InputStreamReader (System.in);
BufferedReader br = new BufferedReader(isr);
String sName;

System.out.print("What is your name?   ");
sName = br.readLine( );
System.out.println("So your name is " + sName);
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Reading integers with BufferedReader

- Reading an int number

```
String sPlaceHolder;
int iAge =0;
int iYear = 2023;

System.out.print("How old are you?  ");
sPlaceHolder = br.readLine( );
iAge = Integer.parseInt(sPlaceHolder);
System.out.println("So you are  " + iAge + " years old");
System.out.println("you were probably born in "+ (iYear - iAge));
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Reading doubles with BufferedReader

- Reading a double number

```
final double dGoal = 1000000;  //constant
double dEarnings = 0;
double dMissing = 0;

System.out.print("How much do you make per hour?  ");
sPlaceHolder = br.readLine( )
dEarnings = Double.parseDouble(sPlaceHolder);
dMissing = dGoal - dEarnings;
System.out.println("\nIn one hours you will only need " + iMissing + "more to earn your goal");
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Scanner syntax

- The Scanner class is found in the java.util package.

  import java.util.*; // so you can use Scanner

- Constructing a Scanner object to read console input:

  Scanner name = new Scanner(System.in);

- Example:

  Scanner sc = new Scanner(System.in);

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Scanner Class -The Methods

- **Constructor**

  Scanner sc = new Scanner(System.in);

- **Reading a String**

  String sc.nextLine( );

- **Reading an int**

  int sc.nextInt( );

- **Reading a double**

  double sc.nextDouble( );

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Scanner methods

Method	Description
nextInt()	reads an integer from the user
nextDouble()	reads a double from the user
next()	reads a one-word String from the user
nextLine()	reads a one-line String from the user

- Each method waits until the user presses Enter.
- The value typed by the user is returned.
- Example:

```
System.out.print("How old are you? ");  // prompt
int age = sc.nextInt();
System.out.println("You typed " + age);
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Reading a String with the Scanner class

- Reading a String

```
Scanner scan = new Scanner(System.in);

String str;

System.out.print("What is your name?   ");
str = scan.nextLine( );
System.out.println("So your name is " + str);
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Reading integers with the Scanner class

- Reading an int number

```
Scanner scan = new Scanner(System.in);

int iAge = 0;
Int iYear = 2023;

System.out.print("How old are you? ");
iAge = scan.nextInt( );
iYear = iYear - iAge;
System.out.println("\nSo you were born around " + iYear);
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Reading doubles with the Scanner Class

- Reading a double number

```
Scanner scan = new Scanner(System.in);

final double dGoal = 1000000;  //constant
double dEarnings = 0;
double dMissing = 0;

System.out.print("How much do you make per hour?  ");
dEarnings = scan.nextDouble(  );
dMissing = dGoal - dEarnings;
System.out.println("\nIn one hours you will only need " + iMissing +
    "more to earn your goal");
```



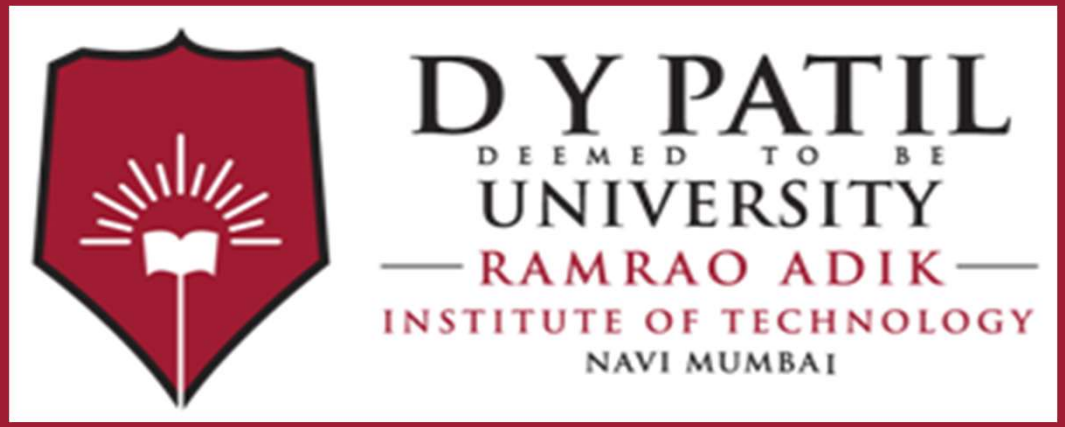
D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Difference between Scanner and BufferReader Class in Java

Sr. No.	Key	Scanner Class	BufferReader Class
1	Synchronous	Scanner is not synchronous in nature and should be used only in single threaded case.	BufferReader is synchronous in nature. During multithreading environment, BufferReader should be used.
2	Buffer Memory	Scanner has little buffer of 1 KB char buffer.	BufferReader has large buffer of 8KB byte Buffer as compared to Scanner.
3	Processing Speed	Scanner is bit slower as it need to parse data as well.	BufferReader is faster than Scanner as it only reads a character stream.
4	Methods	Scanner has methods like nextInt(), nextShort() etc.	BufferReader has methods like parseInt(), parseShort() etc.
5	Read Line	Scanner has method nextLine() to read a line.	BufferReader has method readLine() to read a line.



D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI


D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You